EDHEC Business School

M.Sc. in Financial Engineering

Advanced Derivatives

Coursework Report

Academic Year 2026–27

Group X

#	Name	Student ID
1	Paolo Mastrogiacomo	
2		
3		

Deadline: Tuesday 1 December 2026

Code: `ad_coursework_code.py` (all results reproducible, fixed seeds)

Question 1: Black-Scholes-Merton hedging

1(a) Using the FinancePy library

We call the analytic Black–Scholes functions in `financepy.models.black_scholes_analytic` directly, with the option type passed as an integer flag. In the current FinancePy release (1.1.x) these functions are named `value`, `delta` and `gamma`; older releases call them `bs_value`, `bs_delta`, `bs_gamma`. The import handles both. For $S = K = 100$, $r = 4\%$, $\sigma = 20\%$, $T = 1$ the put is worth 6.0040, with delta -0.3821 and gamma 0.01907.

Code 1: Direct calls into the FinancePy model library.

```
# Question 1(a): call the FinancePy *model* functions directly instead of
# building EquityVanillaOption objects -- this is roughly 60x faster.
# Older FinancePy releases call them bs_value / bs_delta / bs_gamma; the
# current release (1.1.x) calls them value / delta / gamma.  We support both.
try:
    from financepy.models.black_scholes_analytic import (
        bs_value, bs_delta, bs_gamma)
except ImportError:
    from financepy.models.black_scholes_analytic import (
        value as bs_value, delta as bs_delta, gamma as bs_gamma)
from financepy.utils.global_types import OptionTypes

PUT = OptionTypes.EUROPEAN_PUT.value     # integer flag the model functions use

# Example direct calls (S, T, K, r, q, sigma, option type)
put_value = bs_value(100.0, 1.0, 100.0, 0.04, 0.0, 0.20, PUT)
put_delta = bs_delta(100.0, 1.0, 100.0, 0.04, 0.0, 0.20, PUT)
put_gamma = bs_gamma(100.0, 1.0, 100.0, 0.04, 0.0, 0.20, PUT)
```

1(b) The `DeltaHedge` function

At $t_0 = 0$ we receive the Black–Scholes premium V_0, buy Δ_0 shares (negative for a put) and put $B_0 = V_0 - \Delta_0 S_0$ in the bank. On each of the NT dates the stock moves under the *real-world* lognormal dynamics with drift μ, the bank account accrues $e^{r\Delta t}$, and we rebalance to the new delta. Each rebalance is paid for out of the bank, so the portfolio is self-financing. At expiry the replication error is $\Delta_T S_T + B_T - (K - S_T)^+$. The realised variance is $\frac{1}{T}\sum_i \ln^2(S_{i+1}/S_i)$. The assignment says the output has four elements but lists five; we return all five.

Code 2: `DeltaHedge`: self-financing delta hedge of a European put.

```
def DeltaHedge(K, S, r, mu, sigma, T, N, sigma_real=None, q=0.0, rng=None):
    """Delta hedge of a European PUT with a self-financing portfolio.
    mu = real-world drift, sigma = vol used to price/hedge, sigma_real = vol
    of the simulated path (default sigma), N = hedges per year.
    Returns (S_T, payoff, realised_variance, replication_error, avg_gamma),
    replication_error = hedge portfolio value at T - option payoff."""
    rng = np.random.default_rng() if rng is None else rng
    sigma_real = sigma if sigma_real is None else sigma_real

    n_steps = int(round(N * T))            # number of hedging intervals
    dt = T / n_steps
    z = rng.standard_normal(n_steps)       # one shock per interval

    # t = 0: receive the BS premium and buy delta shares (delta < 0 for a put)
    stock = S
    delta = bs_delta(stock, T, K, r, q, sigma, PUT)
    gammas = [bs_gamma(stock, T, K, r, q, sigma, PUT)]
    cash = bs_value(stock, T, K, r, q, sigma, PUT) - delta * stock
    log_returns = np.empty(n_steps)

    for i in range(n_steps):
        # 1) the stock moves under the REAL-world lognormal dynamics
        new_stock = stock * np.exp((mu - 0.5 * sigma_real**2) * dt
                                   + sigma_real * np.sqrt(dt) * z[i])
        log_returns[i] = np.log(new_stock / stock)
        stock = new_stock

        # 2) cash accrues interest; dividends on the shares we hold are received
        cash = cash * np.exp(r * dt) + delta * stock * (np.exp(q * dt) - 1.0)
```

```
        # 3) rebalance to the new delta (not on the expiry date itself)
        tau = T - (i + 1) * dt              # remaining time to expiry
        if i < n_steps - 1:
            new_delta = bs_delta(stock, tau, K, r, q, sigma, PUT)
            cash -= (new_delta - delta) * stock   # self-financing trade
            delta = new_delta
            gammas.append(bs_gamma(stock, tau, K, r, q, sigma, PUT))

    portfolio = delta * stock + cash
    payoff = max(K - stock, 0.0)
    realised_variance = np.sum(log_returns**2) / T   # annualised
    return stock, payoff, realised_variance, portfolio - payoff, np.mean(gammas)
```

1(c) Running 10,000 paths

Code 3: Wrapper that runs any hedging function over many paths and returns a DataFrame.

```
def run_hedge_paths(hedge_fn=None, n_paths=10_000, seed=42, **params):
    """Run a hedging function (default DeltaHedge) on n_paths independent
    paths; returns a DataFrame with one row per path."""
    hedge_fn = DeltaHedge if hedge_fn is None else hedge_fn
    rng = np.random.default_rng(seed)       # same seed -> same shocks
    rows = [hedge_fn(rng=rng, **params) for _ in range(n_paths)]
    df = pd.DataFrame(rows, columns=["S_T", "payoff", "realised_var",
                                     "error", "avg_gamma"])
    df["realised_vol"] = np.sqrt(df["realised_var"])
    return df
```

1(d) Hedging error versus terminal stock price

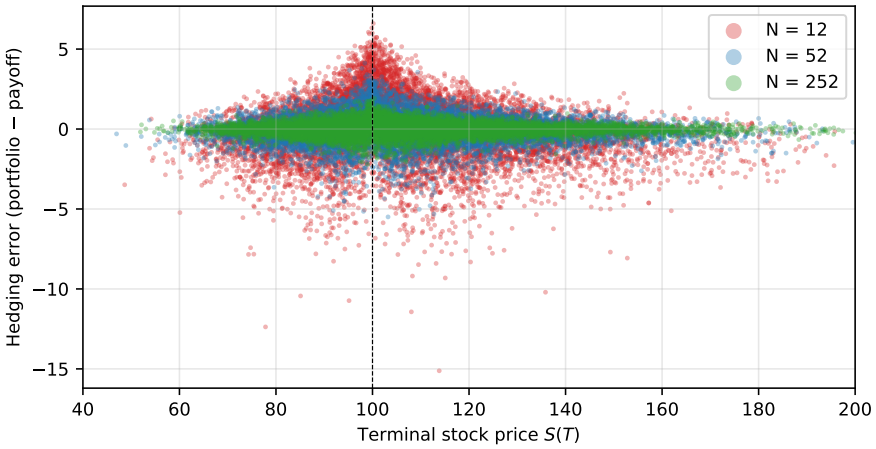


Figure 1: Hedging error against $S(T)$ for $N = 12, 52, 252$. $S(0) = 100$, $K = 100$, $r = 4\%$, $T = 1$, $\sigma = \sigma_{\text{real}} = 20\%$, $\mu = 8\%$, 10,000 paths per N.

The errors form a "spindle" centred on the strike. Paths that end far in or far out of the money have small errors, because there the put's delta is almost constant (-1 or 0) and the hedge barely needs rebalancing. Paths that end near $K = 100$ have the largest errors, because that is where gamma is highest and discrete rebalancing misses the most. The cloud shrinks visibly as N increases.

1(e) Mean and variance of the hedging error

Table 1: Hedging error over 10,000 paths. "Theory std." is the Derman–Kamal approximation $\sqrt{\pi/4}\,\mathcal{V}\sigma/\sqrt{N}$, with $\mathcal{V}$ the option's vega.

N	Mean	Variance	Std. dev.	Theory std.
12	-0.0383	3.8045	1.9505	1.9514
52	-0.0082	0.8986	0.9480	0.9374
252	0.0019	0.1872	0.4327	0.4258

The mean error is essentially zero, so the hedge is unbiased: the premium is exactly enough to finance the replication. The variance falls roughly in proportion to $1/N$: going from monthly to daily hedging cuts the standard deviation from 1.95 to 0.43, about a third of the premium down to 7% of it. The simulated standard deviations match the Derman–Kamal approximation almost exactly.

1(f)–(g) Realised volatility, average gamma and the replication error

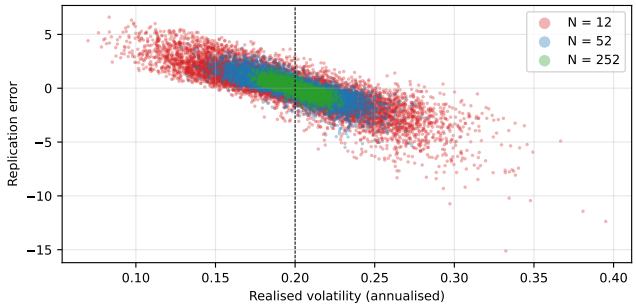


Figure 2: Replication error against realised volatility.

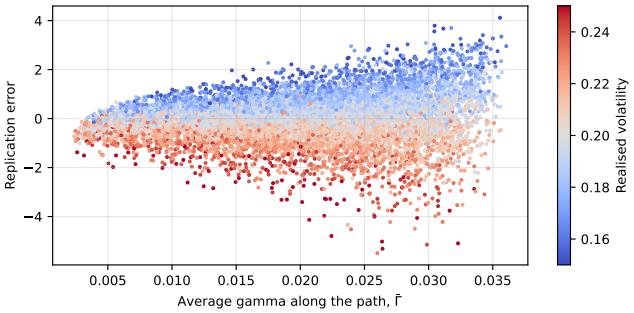


Figure 3: Replication error against average gamma ($N = 52$), coloured by realised volatility.

There is a strong negative relationship: the correlation with realised variance is about -0.81 for every N. A second-order (Taylor) expansion of the hedged position gives

$$\text{error} \;\approx\; \tfrac{1}{2}\sum_i \Gamma_i S_i^2 \left(\sigma^2 - \sigma_{\text{real},i}^2\right) \Delta t.$$

The option is long gamma and the replicating portfolio is not. The premium we received pays for 20% volatility. If the stock actually moved more ($\sigma_{\text{real}} > 20\%$), the option is worth more than our portfolio and the error is negative; if it moved less, we make money. The scatter around the line is the pure discreteness error, which narrows as N grows.

The plot fans out: the higher the average gamma, the larger the absolute error. The error standard deviation is 1.30 for the top quartile of average gamma against 0.54 for the bottom quartile. Gamma is the *size* of the exposure and the volatility surprise $\sigma^2 - \sigma_{\text{real}}^2$ sets its *sign*, as the colours show: low-volatility (blue) paths sit above zero and high-volatility (red) paths below. High-gamma paths are those that stay near the strike. Using only the average gamma, the formula above already has a correlation of 0.87 with the simulated error.

1(h) Drift sensitivity

Table 2: Hedging error for different real-world drifts ($N = 52$, 10,000 paths, the same random numbers for every μ).

μ	Mean error	Mean abs. error	Variance	Mean $S(T)$
2.5%	-0.0051	0.7113	0.9037	102.4356
5.0%	-0.0067	0.7132	0.9080	105.0288
7.5%	-0.0082	0.7098	0.9001	107.6876
10.0%	-0.0109	0.6989	0.8842	110.4137

The drift has a *very small* effect. Raising μ from 2.5% to 10% moves the mean terminal price by 8 points, yet the mean absolute error only goes from 0.711 to 0.699 and the variance from 0.904 to 0.884, which is the same order as the Monte Carlo noise. Delta hedging removes the directional exposure, so the expected return of the stock does not enter the Black–Scholes price or the hedge. The slight improvement at higher μ is because the put then ends further out of the money, where gamma is lower.

Question 2: Time-varying volatility

2(a) A two-period volatility term structure

Table 3: Put values with $S(0) = K = 100$, $r = 4\%$, $T = 1$.

Volatility	$\bar{\sigma}$	Put value	Std. error
Flat 20%	0.2000	6.0040	0.0000
4% then 28% (BS with $\bar{\sigma}$)	0.2000	6.0040	0.0000
4% then 28% (MC, 10^6 paths)	0.2000	6.0081	0.0090

A European payoff depends only on the distribution of $S(T)$. That distribution is lognormal with total variance $\int_0^T \sigma^2(u)\,du$, so only the average variance counts: $\bar{\sigma}^2 = \frac{1}{2}(0.04^2) + \frac{1}{2}(0.28^2) = 0.0008 + 0.0392 = 0.04 = 0.20^2$. The two prices are therefore identical (6.0040). A Monte Carlo simulation of the two-period dynamics confirms this within one standard error.

2(b) `DeltaHedgeTV`

The stock path uses the *instantaneous* volatility: each step has variance $\int_t^{t+\Delta t} \sigma^2(u)du$. Pricing and hedging at time t must use the *average volatility over the remaining life*, $\bar{\sigma}(t) = \sqrt{\int_t^T \sigma^2(u)du/(T-t)}$. This changes as time to maturity falls. With the low volatility first, $\bar{\sigma}(t)$ rises from 20% at $t = 0$ to 28% at $t = \frac{1}{2}$ and then stays there. With the high volatility first, it falls from 20% to 4%.

Code 4: Integrated variance and remaining-life average volatility.

```
def integrated_variance(t1, t2, vols, T):
    """
    Integral of sigma(u)^2 du from t1 to t2 for a piecewise-constant
    volatility: vols[j] applies on the j-th of len(vols) equal sub-periods
    of [0, T].
    """
    edges = np.linspace(0.0, T, len(vols) + 1)
    total = 0.0
    for j, v in enumerate(vols):
        overlap = max(0.0, min(t2, edges[j + 1]) - max(t1, edges[j]))
        total += v**2 * overlap
    return total


def remaining_vol(t, vols, T):
    """Volatility to price with at time t: the root-mean-square (average)
    volatility over the REMAINING life [t, T] of the option."""
    tau = T - t
    return np.sqrt(integrated_variance(t, T, vols, T) / tau)
```

Code 5: `DeltaHedgeTV`.

```python
def DeltaHedgeTV(K, S, r, mu, vols, T, N, q=0.0, rng=None, naive=False):
    """DeltaHedge with piecewise-constant vol (vols on equal sub-periods).
    Path: INSTANTANEOUS variance per step.  Pricing/hedging: AVERAGE vol over
    the REMAINING life.  naive=True (wrongly) hedges with today's spot vol.
    Returns the same 5-tuple as DeltaHedge."""
    rng = np.random.default_rng() if rng is None else rng
    n_steps = int(round(N * T))
    dt = T / n_steps
    z = rng.standard_normal(n_steps)
    edges = np.linspace(0.0, T, len(vols) + 1)

    def hedge_vol(t):
        if naive:   # instantaneous vol at time t
            return vols[min(np.searchsorted(edges, t, side="right") - 1,
                            len(vols) - 1)]
        return remaining_vol(t, vols, T)

    stock, t = S, 0.0
    sig = hedge_vol(t)
    delta = bs_delta(stock, T, K, r, q, sig, PUT)
    gammas = [bs_gamma(stock, T, K, r, q, sig, PUT)]
    cash = bs_value(stock, T, K, r, q, remaining_vol(0.0, vols, T), PUT) \
        - delta * stock                       # premium is always the fair value
    log_returns = np.empty(n_steps)

    for i in range(n_steps):
        var_step = integrated_variance(t, t + dt, vols, T)  # true variance
        new_stock = stock * np.exp(mu * dt - 0.5 * var_step
                                   + np.sqrt(var_step) * z[i])
        log_returns[i] = np.log(new_stock / stock)
        stock, t = new_stock, t + dt
        cash = cash * np.exp(r * dt) + delta * stock * (np.exp(q * dt) - 1.0)
        if i < n_steps - 1:
            tau = T - t
            sig = hedge_vol(t)
            new_delta = bs_delta(stock, tau, K, r, q, sig, PUT)
            cash -= (new_delta - delta) * stock
            delta = new_delta
            gammas.append(bs_gamma(stock, tau, K, r, q, sig, PUT))

    portfolio = delta * stock + cash
    payoff = max(K - stock, 0.0)
    return (stock, payoff, np.sum(log_returns**2) / T, portfolio - payoff,
            np.mean(gammas))
```

2(c) Hedging performance with time-varying volatility

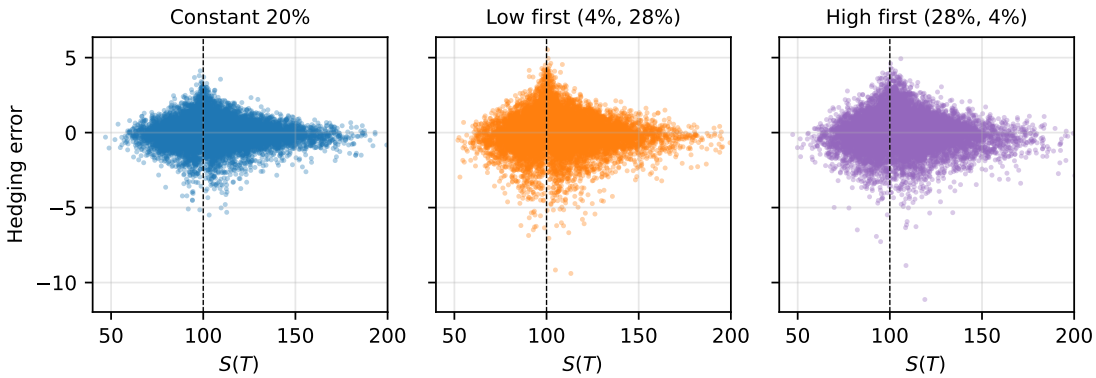


Figure 4: Hedging error against $S(T)$, $N = 52$, 10,000 paths, the same random numbers in all panels.

Table 4: Hedging error: constant versus time-varying volatility (4% then 28%), 10,000 paths.

Case	N	Mean	Variance	Mean abs.	Avg. $\bar{\Gamma}$
Constant 20%	12	-0.0383	3.8045	1.4672	0.0191
Constant 20%	52	-0.0082	0.8986	0.7090	0.0192
Constant 20%	252	0.0019	0.1872	0.3223	0.0192
Low first (4%, 28%)	12	-0.0519	7.0011	2.0095	0.0185
Low first (4%, 28%)	52	-0.0168	1.6906	0.9658	0.0185
Low first (4%, 28%)	252	0.0010	0.3503	0.4400	0.0185

With the correct $\bar{\sigma}(t)$ the hedge is still unbiased (mean ≈ 0), but the error variance almost *doubles* at every N (1.69 against 0.90 for $N = 52$). Hedging with today's instantaneous volatility instead gives a large bias and four times the variance (mean +0.54, variance 3.90 at $N = 52$).

2(d) Reversing the volatility structure

Table 5: Hedging error when the high-volatility period comes first (28% then 4%), 10,000 paths.

Case	N	Mean	Variance	Mean abs.	Avg. $\bar{\Gamma}$
High first (28%, 4%)	12	-0.0604	6.8868	1.9902	0.0192
High first (28%, 4%)	52	0.0042	1.5827	0.9530	0.0196
High first (28%, 4%)	252	0.0035	0.3255	0.4324	0.0198

2(e) Comparing the three cases

- **Price.** All three cases have the same total variance, so the premium is the same and, with the correct remaining-life volatility, the mean hedging error is zero in all three.
- **Risk.** Both time-varying cases have roughly twice the error variance of the constant case. Each step's error is roughly $\frac{1}{2}\Gamma S^2(\Delta S^2/S^2 - \sigma_t^2\Delta t)$, whose variance grows with σ_t^4. The total error variance therefore depends on $\int \sigma^4\,dt$, not on $\int \sigma^2\,dt$. With constant volatility $\sigma^4 = 0.0016$; for the two-period cases the average $\frac{1}{2}(0.04^4 + 0.28^4) = 0.0031$ is 1.92 times larger, close to the observed ratios (1.84–1.88 for low-first, 1.74–1.81 for high-first). The same variance delivered in fewer, larger moves is harder to hedge discretely.
- **Ordering.** Low-volatility-first is slightly riskier than high-volatility-first (1.69 against 1.58 at $N = 52$). With the quiet period first, the stock is still close to the strike when the volatile period starts, and those large moves then happen close to expiry, where delta changes fastest for at-the-money paths. With the volatile period first, the large moves happen while the option still has a long time to run; by the time expiry approaches the stock has usually moved away from the strike and barely moves.
- **Main lesson.** Using the right volatility matters much more than the ordering. The naive hedge with the instantaneous volatility is biased (+0.54) and has a variance of 3.90.

Question 3: Asian options and the control variate technique

3(a) Pricing an arithmetic average Asian call

The stock is simulated under the risk-neutral measure. One step goes from 0 to $T_0 = 0.25$, because nothing is averaged before T_0. Then 90 daily steps of $\Delta t = 0.25/90$ go to $T = 0.5$, and $\bar{S}$ is the average over those 90 dates. The price is the mean of the discounted payoffs $e^{-rT}(\bar{S} - K)^+$. All paths are generated at once with NumPy.

Code 6: Path simulation and arithmetic Asian call pricer.

```
404 def simulate_averaging_prices(S0, r, q, sigma, T, T0, n_obs, P, rng):
405     """Risk-neutral GBM prices on the averaging dates T0+dt, ..., T: one jump
406     from 0 to T0, then n_obs daily steps.  Returns a (P x n_obs) array."""
```

```
    dt = (T - T0) / n_obs
    drift = r - q - 0.5 * sigma**2
    z0 = rng.standard_normal(P)                    # shock over [0, T0]
    z = rng.standard_normal((P, n_obs))            # daily shocks
    log_s_T0 = np.log(S0) + drift * T0 + sigma * np.sqrt(T0) * z0
    log_s = log_s_T0[:, None] + np.cumsum(drift * dt + sigma * np.sqrt(dt) * z,
                                          axis=1)
    return np.exp(log_s)


def asian_arithmetic_mc(S0, K, r, sigma, T=0.5, T0=0.25, n_obs=90, P=10_000,
                        q=0.0, seed=1):
    """Q3(a): Monte-Carlo price of an arithmetic-average Asian CALL.
    Returns (price, standard error)."""
    rng = np.random.default_rng(seed)
    prices = simulate_averaging_prices(S0, r, q, sigma, T, T0, n_obs, P, rng)
    arith_avg = prices.mean(axis=1)
    disc_payoff = np.exp(-r * T) * np.maximum(arith_avg - K, 0.0)
    return disc_payoff.mean(), disc_payoff.std(ddof=1) / np.sqrt(P)
```

3(b) A first price

For $S(0) = K = 100$, $r = 4\%$, $\sigma = 25\%$ and $P = 10{,}000$ paths, the price is **6.4185** with a standard error of 0.0974. The 95% confidence interval is [6.23, 6.61].

3(c)–(d) Adding the geometric average; variances and covariance

Code 7: Arithmetic and geometric Asian calls on the same paths; returns the five requested numbers.

```
def asian_mc(S0, K, r, sigma, T=0.5, T0=0.25, n_obs=90, P=10_000, q=0.0,
             seed=1, return_payoffs=False):
    """Arithmetic AND geometric Asian calls on the SAME paths.  Returns
    [geo price, arith price, var geo payoff, var arith payoff, covariance]
    (per-path variances of discounted payoffs; estimate variance = var/P)."""
    rng = np.random.default_rng(seed)
    prices = simulate_averaging_prices(S0, r, q, sigma, T, T0, n_obs, P, rng)
    arith_avg = prices.mean(axis=1)
    geo_avg = np.exp(np.log(prices).mean(axis=1))
    disc = np.exp(-r * T)
    pay_a = disc * np.maximum(arith_avg - K, 0.0)
    pay_g = disc * np.maximum(geo_avg - K, 0.0)
    cov = np.cov(pay_g, pay_a, ddof=1)             # 2x2 covariance matrix
    out = np.array([pay_g.mean(), pay_a.mean(), cov[0, 0], cov[1, 1],
                    cov[0, 1]])
    if return_payoffs:
        return out, pay_g, pay_a, prices
    return out
```

3(e) Numerical results

Table 6: $S(0) = K = 100$, $r = 5\%$, $q = 1\%$, $\sigma = 30\%$, $T = 0.5$, $T_0 = 0.25$, 90 days, $P = 5{,}000$. Variances and covariance are per path for the discounted payoffs.

Quantity	Value
Geometric price	7.2391
Arithmetic price	7.3440
Var. disc. geometric payoff	132.1295
Var. disc. arithmetic payoff	134.5193
Covariance	133.3068
Std. error geometric price	0.1626
Std. error arithmetic price	0.1640

The two payoffs have almost the same variance and a covariance close to both, so their correlation is close to one. The geometric average is always below the arithmetic one, so the geometric call is the cheaper option.

3(f) The control variate estimate

We read "$T_0 = 0$" as averaging that starts today and runs daily (180 days) to expiry. The second row of Table 7 applies the same method to the original contract with $T_0 = 0.25$. The discretely sampled geometric average is exactly lognormal, so its price follows from a Black formula (Code 8). The best (minimum-variance) estimator is

$$\hat{A}_{\mathrm{CV}} = \bar{A} - \beta\left(\bar{G} - G^{\mathrm{exact}}\right), \qquad \beta = \frac{\mathrm{Cov}(A,G)}{\mathrm{Var}(G)}, \qquad \mathrm{Var}(\hat{A}_{\mathrm{CV}}) = \frac{\mathrm{Var}(A)(1-\rho^2)}{P}.$$

Table 7: Control variate results ($P = 5{,}000$). K–V is the continuous-averaging Kemna–Vorst formula from the lecture.

Contract	G exact	G K–V	G MC	A MC	s.e.	A CV	s.e. CV	β	ρ
$T_0 = 0$ (180 days)	5.1121	5.0893	5.0970	5.3098	0.1178	5.3255	0.0032	1.0379	0.9996
$T_0 = 0.25$ (90 days)	7.4562	7.4390	7.2391	7.3440	0.1640	7.5629	0.0022	1.0089	0.9999

The best estimate of the arithmetic Asian price is **5.3255** (standard error 0.0032) for $T_0 = 0$, and 7.5629 (0.0022) for the original contract. The control variate cuts the standard error by a factor of 35 to 75. The geometric price must be the *discrete* formula: the continuous Kemna–Vorst value differs by about 0.02, which would bias the estimate.

Code 8: Exact discrete geometric Asian price and delta, and the control variate estimator.

```
def geometric_asian_exact(S0, K, r, sigma, T, T0, n_obs, q=0.0):
    """Exact price and delta of the DISCRETE geometric Asian call: log G is
    normal, mean m = log S0 + (r-q-sigma^2/2) mean(t_i),
    variance v = sigma^2/n^2 sum_ij min(t_i, t_j) -> Black formula."""
    t = T0 + (T - T0) / n_obs * np.arange(1, n_obs + 1)
    m = np.log(S0) + (r - q - 0.5 * sigma**2) * t.mean()
    v = sigma**2 * np.minimum.outer(t, t).sum() / n_obs**2
    d1 = (m - np.log(K) + v) / np.sqrt(v)
    d2 = d1 - np.sqrt(v)
    fwd_g = np.exp(m + 0.5 * v)                    # E[G]
    price = np.exp(-r * T) * (fwd_g * norm.cdf(d1) - K * norm.cdf(d2))
    delta = np.exp(-r * T) * fwd_g * norm.cdf(d1) / S0
    return price, delta


def kemna_vorst(S0, K, r, sigma, T, T0, q=0.0):
    """Continuous-averaging geometric Asian call (Kemna & Vorst / lecture 4)."""
    M = np.log(S0) + (r - q - 0.5 * sigma**2) * (T0 + 0.5 * (T - T0))
    V = sigma**2 * (T0 + (T - T0) / 3.0)
    d1 = (M - np.log(K) + V) / np.sqrt(V)
    return np.exp(-r * T) * (np.exp(M + 0.5 * V) * norm.cdf(d1)
                             - K * norm.cdf(d1 - np.sqrt(V)))


def control_variate(stats, geo_exact):
    """Optimal control-variate estimate of the arithmetic price from the
    5 outputs of asian_mc and the exact geometric price.
    A_cv = A_bar - beta (G_bar - G_exact),  beta = Cov(A,G) / Var(G).
    Returns (estimate, per-path variance of the CV estimator, beta, rho)."""
    g, a, var_g, var_a, cov = stats
    beta = cov / var_g
    est = a - beta * (g - geo_exact)
    var_cv = var_a - cov**2 / var_g                # = var_a * (1 - rho^2)
    rho = cov / np.sqrt(var_g * var_a)
    return est, var_cv, beta, rho
```

3(g) Paths needed with the control variate

The variance of an estimate is the per-path variance divided by P. Matching the variance of a plain simulation with $P = 10{,}000$ paths therefore requires $P_{\mathrm{CV}} = 10{,}000 \cdot \mathrm{Var}_{\mathrm{CV}} / \mathrm{Var}_{\mathrm{plain}} = 10{,}000\,(1-\rho^2)$.

Table 8: Per-path variances and the equivalent number of paths (estimated with 10,000 paths).

Setting	Var plain	Var CV	ρ	Reduction	Paths needed
3(b): $r = 4\%$, $\sigma = 25\%$, $T_0 = 0.25$	95.4274	0.0101	0.9999	9454.0628	2
3(f): $r = 5\%$, $q = 1\%$, $\sigma = 30\%$, $T_0 = 0$	64.6395	0.0517	0.9996	1250.7115	8

Because $\rho > 0.999$, in theory only a handful of paths (2–8) give the same precision as 10,000 plain paths. In practice you still need a few hundred paths to estimate β reliably and for the normal approximation to hold. Even so, the saving is a factor of more than 1,000.

Question 4: The delta of the Asian option

We use the contract from 3(e): $S(0) = K = 100$, $r = 5\%$, $q = 1\%$, $\sigma = 30\%$, $T = 0.5$, averaging daily over the last 90 days, $P = 10{,}000$.

4(a) Finite-difference delta

Code 9: Forward finite-difference delta.

```
def asian_fd_delta(h, S0=100.0, P=P4_PATHS, seed=21, common=True):
    """Forward FD delta (f(S0+h) - f(S0)) / h.  common=True re-uses the SAME
    random numbers (same seed) for both prices; False uses independent ones."""
    base = asian_mc(S0, P=P, seed=seed, **P4)[1]
    bumped = asian_mc(S0 + h, P=P, seed=seed if common else seed + 1000,
                      **P4)[1]
    return (bumped - base) / h
```

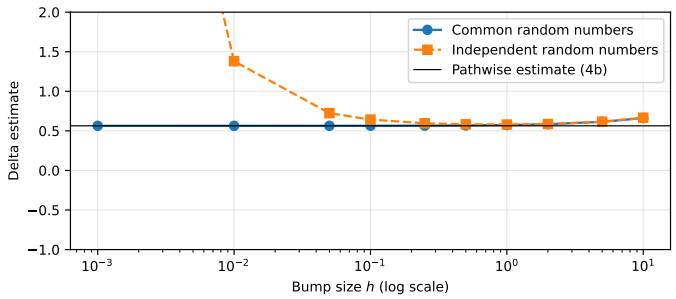


Figure 5: Finite-difference delta against the bump size h.

Table 9: Finite-difference delta estimates.

h	Common RN	Independent RN
0.0010	0.5637	8.7625
0.0100	0.5638	1.3798
0.0500	0.5640	0.7239
0.1000	0.5645	0.6427
0.2500	0.5658	0.5955
0.5000	0.5685	0.5824
1.0000	0.5743	0.5799
2.0000	0.5843	0.5867
5.0000	0.6141	0.6159
10.0000	0.6615	0.6660

We reuse *exactly the same random numbers*, through the same seed and therefore the same normal draws, for $f(S_0)$ and $f(S_0 + h)$ (common random numbers). Every simulated price is proportional to S_0, so the bumped paths are the base paths scaled by $(1 + h/S_0)$. The Monte Carlo noise then cancels in the difference, and the estimate is stable and converges to the pathwise value 0.5637 as $h \to 0$. For large h the forward difference is biased upwards by about $\frac{1}{2}\Gamma h$. With *independent* random numbers the noise of the two prices is divided by h and explodes (8.76 for $h = 0.001$). A bump of about 0.01–0.1 with common random numbers is a good choice.

4(b) Pathwise delta

Code 10: Pathwise delta estimator (no control variate).

```
def asian_pathwise_delta(S0=100.0, P=P4_PATHS, seed=21):
    """Pathwise delta samples exp(-rT) 1(avg > K) avg / S0 for the arithmetic
    and geometric averages (prices are proportional to S0)."""
    _, _, _, prices = asian_mc(S0, P=P, seed=seed, return_payoffs=True, **P4)
    disc = np.exp(-P4["r"] * P4["T"])
    a_bar = prices.mean(axis=1)
    g_bar = np.exp(np.log(prices).mean(axis=1))
    d_arith = disc * (a_bar > P4["K"]) * a_bar / S0
    d_geo = disc * (g_bar > P4["K"]) * g_bar / S0
```

```
    return d_arith, d_geo


d_arith, d_geo = asian_pathwise_delta()
delta_pw = d_arith.mean()
delta_pw_se = d_arith.std(ddof=1) / np.sqrt(P4_PATHS)
```

Delta is estimated as the sample mean of $e^{-rT}\mathbb{I}(\bar{S} > K)\bar{S}/S(0)$ over the paths. The pathwise delta is **0.5637** with a standard error of 0.0057. It needs no bump and only one simulation.

4(c) Control variate for the delta

The control is the pathwise delta of the geometric option, $e^{-rT}\mathbb{I}(G > K)G/S(0)$. Its expectation is known analytically: $e^{-rT}\,\mathbb{E}[G]\,\Phi(d_1)/S(0)$ (Code 8).

Table 10: Plain versus control variate estimates, $P = 10{,}000$. "Var. reduction" is $1/(1-\rho^2)$.

	Plain	s.e.	CV	s.e. CV	ρ	Var. reduction
Price	7.4915	0.1150	7.5626	0.0014	0.9999	6554.6483
Delta	0.5637	0.0057	0.5633	0.0006	0.9937	79.6447

The control variate still helps for the delta: the standard error falls from 0.0057 to 0.0006, equivalent to about 80 times more paths. It is, however, *far less effective* than for the price, where the variance falls by a factor of about 6,500. The delta payoff contains the indicator $\mathbb{I}(\bar{S} > K)$, which jumps. On the 0.4% of paths where the arithmetic and geometric averages fall on opposite sides of the strike, one indicator is 1 and the other is 0. That drops the correlation from 0.9999 to 0.994, and since the variance reduction is $1/(1-\rho^2)$, it is very sensitive to how close ρ is to one.